Proceso de compilación

Federico Velásquez de Luna
Arquitectura Computacional
Ruben Hernández Torres
Ingenieria en Sistemas Computacionales
Universidad Autónoma de Aguascalientes

***Abstract*- En este reporte se llevo a cabo un código en C para generar la serie de Fibonacci. Después se analizaron las diferentes etapas por las que pasa el proceso de compilación: código fuente preprocesamiento, compilación, ensamblado, enlace y ejecutable. Así mismo, se revisaron algunas instrucciones que están el código ensamblador.**

1. INTRODUCCIÓN

Los lenguajes de programación como C son lenguajes de alto nivel, esto quiere decir que es mas fácil leer y entender para los humanos. Pero más allá de eso, este lenguajes es de la categoría de los lenguajes compilados. Esto quiere decir que el código C se transforma en en código máquina especifico del sistema, permitiendo así un alto rendimiento.
Este proceso es realizado mediante el uso de compiladores, cuya tarea es transformar el código de alto nivel en código de bajo nivel (lenguaje ensamblador, código objeto o código máquina) perseverando la funcionalidad del programa.
Este proceso de compilación lleva ciertas etapas para finalmente ejecutar el programa. Tales como: preprocesamiento, compilación, ensamblado, enlace y ejecutable.
En esta práctica se analiza cada una de las etapas usando como ejemplo un programa en lenguaje C, el cual genera la serie de Fibonacci. Se usaron distintas opciones del compilador GCC para detener y observar el resultado de cada etapa.

2. METODOLOGÍA

Para esta el desarrollo de esta práctica, se analizaron las diferentes etapas para poder compilar un código, en este caso, lenguaje C. Se utilizo un entorno de sistema operativo Ubuntu Linux y el compilador GCC (GNU Compiler Collection, versión 15.2.0).

El procedimiento de la práctica se se estructuro de la siguiente manera:

1. Desarrollo del programa
Se redacto un programa en C, el cual tiene como objetivo generar los primeros 10 números de la seria de Fibonacci mediante un ciclo for. Con el final de demostrar las etapas en cada de la compilación, el código se estructuro de la siguiente manera:

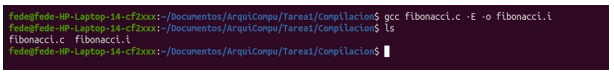
- Directivas del preprocesador para observar la inclusión de librerías estándar.
- Variables locales y estructuras de control para revisar las traducción de instrucciones y manejo de registros en lenguaje ensamblador.

2. Ejecución de los comandos para las etapas de compilación
Se generaron archivos intermedios que corresponde a cada etapa, mediante el uso de los siguientes comando en la terminal:

- Preprocesamiento: se utilizó el comando

  gcc fibonacci.c -E -o fibonacci.i

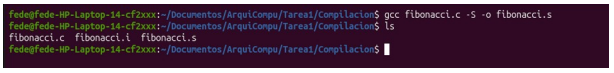
  Se utiliza la bandera -E. El código pasa primero por el preprocesador, el cual lo expande. Se obtiene el archivo fibonacci.i.



- Compilador: se utilizó el comando

  gcc fibonacci.c -S -o fibonacci.s

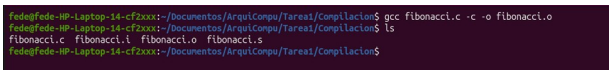
  Se utiliza la bandera -S. El código expandido pasa por el compilador. El cual convierte el código al lenguaje ensamblador respectivo de la arquitectura del procesador. Se obtiene el archivo fibonacci.s.



- Ensamblado: se utilizó el comando

  gcc fibonacci.c -c -o fibonacci.o

  Se utiliza la bandera -c. El código ensamblador se convierte en código objeto. Obteniendo el archivo fibonacci.o.

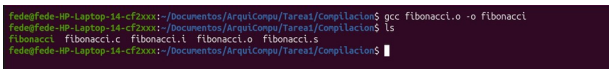


- Enlazado: se utiliza el comando:

  gcc fibonacci.o -o fibonacci

  la función del enlazador es enlazar el código objeto del programa con el código objeto de los archivos de la biblioteca y otros archivos.

Teniendo como resultado el archivo ejecutable: fibonacci.exe.



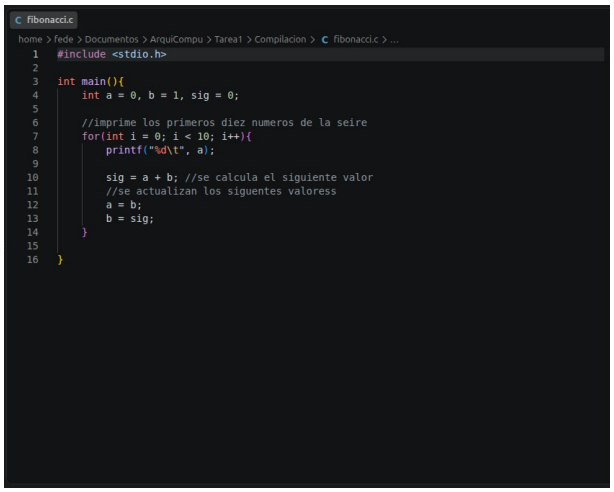
3. Revisión de resultados intermedios

Para verificar la estructura de cada archivo que se nos fue generando después de linea de comando, empleamos el uso del editor de texto *nano*, el cual manejamos desde la terminal. En cada uno de los archivos inspeccionamos el contenido de cada uno para analizar la estructura que emplea cada etapa.

3. CÓDIGOS Y RESULTADOS

A continuación vamos a ver con más detalle el código en lenguaje C, ademas de los comandos con sus banderas en la terminal para su respectiva etapa del proceso de compilación.

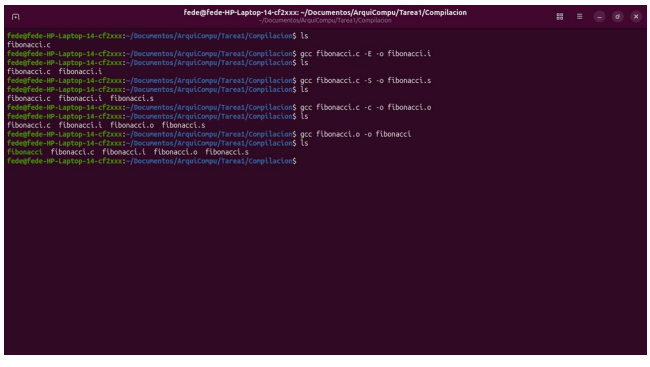
1. Código fuente



En este código, podemos observar un algoritmo simple sobre la serie de fibonacci, el cual imprime los primero 10 número de la serie. Inicia con a en 0 y b en 1. Con un ciclo for que se repite 10, imprime el valor de a, para después calcular el siguiente número de la serie sumando a+b y se guarda en sig. Al final, se actualizan los valores a y b para continuar con los siguientes números de la serie. De esta manera, se obtiene cada término sumando los dos anteriores.

2. Archivos por cada etapa de compilación

Veremos como se ejecuta cada banderilla en la linea de comandos de la terminal.

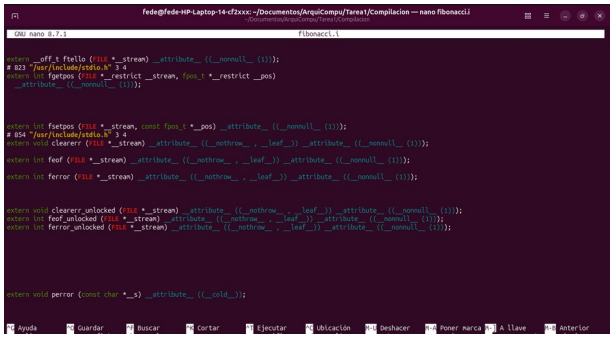


El captura podemos observar que primero mostramos lo que contiene la carpeta mediante el comando *ls*, en este caso, tiene nuestro código en lenguaje C.

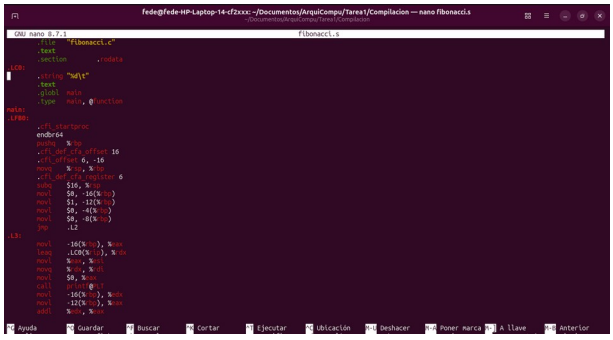
Posteriormente, aplicamos el primer comando para la etapa de preprocesamiento: *gcc fibobanni.c -E -o fibonacci.i.* Luego volvemos a mostrar lo que contiene la carpeta para confirmar que se haya realizado la operación correctamente. Este proceso se lleva a cabo para cada etapa del proceso de la compilación hasta llegar a la ultima parte, donde podemos observar que, al mostrar la carpeta por ultima vez, aparece un archivo en verde, lo cual significa que nuestro ejecutable se ha generado de manera correcta.

Ahora podemos observar algunos de que lleva adentro cada diferente archivo:

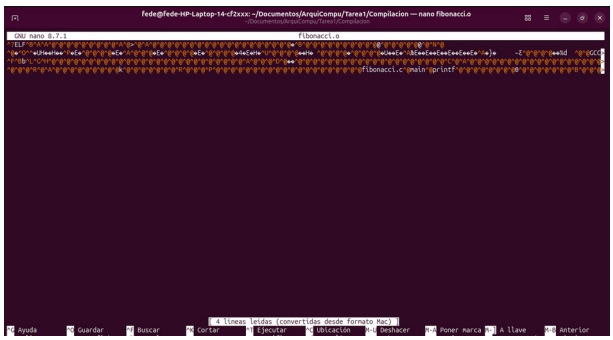
Para el archivo fibonacci.i (Preprocesamiento):



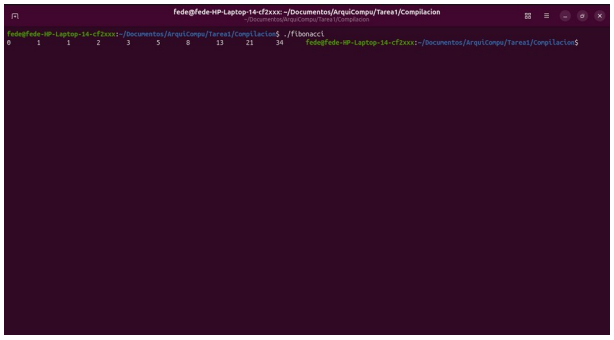
Para el archivo fibonacci.s (Compilación):



Para el archivo fibonacci.o (Ensamblador):

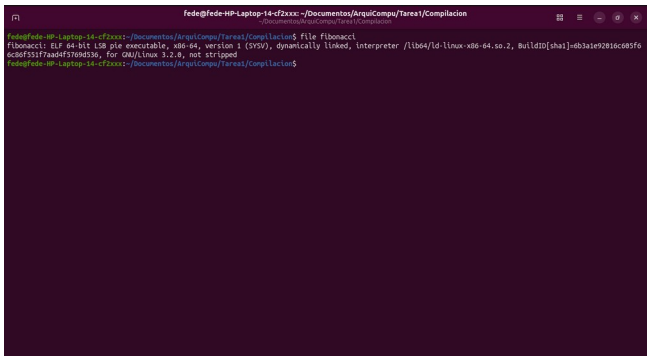


Y por último tenemos el archivo ejecutable:



3. Análisis del ejecutable con el comando *file*

Si ejecutamos el comando en *file fibonacci* en la terminal de comandos, podemos observar que nos proporciona lo siguiente:



La función *file* tiene como objetivo identificar el tipo y mostrar algunas de las características del archivo que se ha mandado como argumento. Así mismo, hace pruebas para clasificar el archivo.

Podemos observar el acrónimo EFL, el significa *Executable and Linking Format*, y es el formato que se utiliza para archivos ejecutables en sistemas Unix y Linux. Además, el 64-bit que lo acompaña, quiere decir que pertenece a la clase EFL de 64 bits. Por otra parte, se observa x86-64 lo cual identifica la arquitectura para la cual fue construido el ejecutable.

4. Instrucciones en lenguaje ensamblador

Como vimos anteriormente, en la etapa de compilación se genera un archivo .s. El cual contiene código de lenguaje de alto nivel traducido a lenguaje máquina para que así el procesador lo pueda ejecutar.

Al abrir ella archivo, podemos ver muchas instrucciones bastantes que una persona normal no podría entender. Y para descifrar algunas de las lineas que hay, podemos revisar el manual de Intel: “Intel 64 and IA Architectures Software Developer’s Manual”. Aquí se describe la arquitectura de procesadores y de entorno de programación que admiten las arquitecturas de los procesadores de Intel.
Vamos a ver de ejemplo algunas instrucciones que están presentes en el archivo .i, y cual es su respectivo significado en el manual.

1) Jmp:

```
movl    $0, -4(%rbp)
movl    $0, -8(%rbp)
jmp     .L2
```

Aquí tenemos la instrucción “jump”, la cual, según el manual, significa lo siguiente:

JMP	Jump.

Esta instrucción pertenece al grupo *5.1.7 Control Transfer Instructions* del manual de arquitecura.
Esta instrucción tiene la función incondicional de darle el control a otro punto del programa. En este caso, a ña etiqueta .L2.

2) Movq:

```
movq    %rdx, %rdi
```

Según el manual, esta instrucción significa:

MOVQ	Move quadword.

Esta instrucción pertenece a la sección *5.4.1 MMX Data Transfer Instructions* del manual de arquitectura.
Su función principal es copiar datos de 64 bits desde un origen a un destino. Donde *mov* indica la operación de mover o copiar datos. Y *q* (quadword) especifica el tamaño del dato. En la arquitectura x86, una “word” es de 16 bits. Por lo tanto, “quadword” es de 64 bits (8 bytes).

3) Call:

```
call    printf@PLT
```

Según el manual, esta instrucción significa:

CALL	Call procedure.

Esta instrucción pertenece a la sección *5.1.7 Control Transfer Instructions* del manual de usuario.
Esta función tiene el objetivo de pasar el control del programa a un procedimiento. Cuando este procedimiento termina, se procesan las instrucciones que siguen de *Call.*

4) Leave:

```
leave
```

Según el manual, esta instrucción significa:

LEAVE	High-level procedure exit.

Esta instrucción pertenece a la sección *5.1.7 Control Transfer Instructions* manual de usuario.
Esta función nos dice que es una salida del procedimiento de alto nivel. Tiene la función de liberar el marco de la pila establecido por una instrucción ENTER anterior.

5) Jle:

```
jle     .L3
```

Según el manual, esta instrucción significa:

JLE/JNG	Jump if less or equal/Jump if not greater.

Esta instrucción pertenece a la sección *5.1.7 Control Transfer Instructions* del manual de arquitectura.
En este caso, la instrucción es *Jump if Less or Equal*, que significa “Salta si menor o igual”. Se encarga de evaluar una instrucción previa, normalmente es un comando de comparación como *cmp*.

4. CONCLUSIONES

En materias anteriores, estuvimos programando en lenguaje C. Cuando realizamos alguna práctica, solo con presionar una teclado bastaba para obtener el archivo ejecutable y después observar nuestro código funcionar. Pero en esta práctica me pude dar cuenta del procedimiento que hay detrás a la hora de compilar un código. Pude revisar como el código en lenguaje de alto nivel pasa por cada una de la etapas pasa en convertirse en lenguaje ensamblador y posteriormente en código máquina en el archivo objeto y ejecutable.

La generación de archivos para cada etapa diferente del proceso me permitió observar mas detalladamente que es lo que pasaba en cada una de estas, dejándolo de ver como un proceso automático. Así mismo, me pude dar cuenta de lo importante que es conocer la arquitectura del procesador, ya que para poder analizar el código ensamblador es necesario revisar el manual para conocer como funcionan las instrucciones que se utilizaron y como el procesador las representa para hacer operaciones.

Esta práctica me dejo comprender más como el código que escribimos en nuestro día a día, es procesado hasta convertirse en instrucciones que puede entender la computadora.

5. REFERENCIAS

[1] IBM, “¿Qué es un compilador?”. [Online]. Available: https://www.ibm.com/mx-es/think/topics/compiler

[2] Medium, “pasos de compilación”. [Online]. Available: https://medium.com/@3681/steps-of-compilation-5c02935a3904

[3] MalwareIntel Blog, “Formato ELF de Linux: Estructura y Análisis para Malware”. [Online]. Available: https://malwareintel.es/blog/analisis-binarios/formato-elf-linux-analisis/

[4] O’Really, “El arte del lenguaje ensamblador, 2.ª edición”. [Online]. Avaliable: https://www.oreilly.com/library/view/the-art-of/9781593272074/ch07s03.html

[5] IBM, “Instrucciones CALL y RETURN”. [Online]. Avaliable: https://www.ibm.com/docs/es/cics-ts/5.6.0?topic=instructions-call-return

[6] Felix Cloutier, “LEAVE – High Level Procedure Exit”. [Online]. Avaliable: https://www.felixcloutier.com/x86/leave

Link para repositorio de Github: https://github.com/FedeMoon/Arquitectura-Computacional.git